**IC TRAINING CENTER VIET NAM**

**FUNDAMENTAL IC DESIGN AND VERIFICATION COURSE**




# ĐỒ ÁN CUỐI KHÓA

# ĐẶC TẢ THIẾT KẾ TIMER IP

## (Mức Advanced)

**Học viên: Ngô Hồ Quân**

**Lớp: IC43**

**Giảng viên: Phạm Hữu Khang**

# Mục lục

1. Tổng quan . . . . . 4
   1.1. Giới thiệu . . . . . 4
   1.2. Tính năng chính . . . . . 4
   1.3. Sơ đồ khối . . . . . 5
   1.4. Tín hiệu giao tiếp . . . . . 5

2. Đặc tả thanh ghi . . . . . 6
   2.1. Tổng hợp thanh ghi . . . . . 6
   2.2. Timer Control Register (TCR) . . . . . 6
   2.3. Timer Data Register 0 (TDR0) . . . . . 7
   2.4. Timer Data Register 1 (TDR1) . . . . . 7
   2.5. Timer Compare Register 0 (TCMP0) . . . . . 8
   2.6. Timer Compare Register 1 (TCMP1) . . . . . 8
   2.7. Timer Interrupt Enable Register (TIER) . . . . . 9
   2.8. Timer Interrupt Status Register (TISR) . . . . . 9
   2.9. Timer Halt Control Status Register (THCSR) . . . . . 10

3. Mô tả chức năng . . . . . 10
   3.1. APB slave . . . . . 11
   3.2. Register . . . . . 15
   3.3. Counter control . . . . . 22
   3.4. Counter . . . . . 25
   3.5. Interrupt . . . . . 28

4. Lịch sử . . . . . 30

## DANH SÁCH BẢNG

Bảng 1: Tín hiệu giao tiếp của Timer IP . . . 5
Bảng 2: Tổng hợp thanh ghi . . . 6
Bảng 3: Timer Control Register (TCR) . . . 7
Bảng 4: Timer Data Register 0 (TDR0) . . . 7
Bảng 5: Timer Data Register 1 (TDR1) . . . 8
Bảng 6: Timer Compare Register 0 (TCMP0) . . . 8
Bảng 7: Timer Compare Register 1 (TCMP1) . . . 9
Bảng 8: Timer Interrupt Enable Register (TIER) . . . 9
Bảng 9: Timer Interrupt Status Register (TISR) . . . 9
Bảng 10: Timer Halt Control Status Register (THCSR) . . . 10
Bảng 11: Tín hiệu nội bộ và sơ đồ tạo ra chúng . . . 10
Bảng 12: Điều kiện tạo error response (ghi TCR) . . . 13

## DANH SÁCH HÌNH

Hình 1: Sơ đồ khối của Timer IP . . . 5
Hình 2: Dạng sóng ghi APB slave (wait state, byte access, error response) . . . 12
Hình 3: Dạng sóng đọc APB slave (wait state) . . . 13
Hình 4: Sơ đồ logic APB slave . . . 14
Hình 5: Dạng sóng ghi/đọc thanh ghi . . . 15
Hình 6: Sơ đồ logic giải mã địa chỉ . . . 16
Hình 7: Sơ đồ logic phát hiện lỗi truy cập TCR (wr_err) . . . 16
Hình 8: Sơ đồ logic thanh ghi TCR . . . 17
Hình 9: Sơ đồ logic thanh ghi TCMP0 . . . 18
Hình 10: Sơ đồ logic thanh ghi TCMP1 . . . 19
Hình 11: Sơ đồ logic tdr0_wr_en/tdr1_wr_en . . . 20
Hình 12: Sơ đồ logic int_st_clr . . . 20
Hình 13: Sơ đồ logic thanh ghi TIER (INT_EN) . . . 20
Hình 14: Sơ đồ logic thanh ghi THCSR (HALT_REQ) . . . 21
Hình 15: Sơ đồ logic mux đọc . . . 21
Hình 16: Dạng sóng chế độ đếm . . . 23
Hình 17: Dạng sóng halt trong debug mode (DIV_VAL = 2) . . . 23
Hình 18: Sơ đồ logic halted/halt_ack . . . 24
Hình 19: Sơ đồ logic bộ đếm chia div_cnt . . . 24
Hình 20: Sơ đồ logic cnt_en . . . 25
Hình 21: Sơ đồ logic phát hiện cạnh xuống TIM_EN (cnt_clr) . . . 25
Hình 22: Dạng sóng bộ đếm . . . 26
Hình 23: Sơ đồ logic dữ liệu ghi bộ đếm cnt_wdata . . . 27
Hình 24: Sơ đồ logic bộ đếm 64-bit . . . 28
Hình 25: Dạng sóng ngắt timer . . . 29
Hình 26: Sơ đồ logic khối interrupt (INT_ST, tim_int) . . . 29

# 1. Tổng quan

## 1.1. Giới thiệu

Timer IP (module top: timer_top) được tùy biến từ module CLINT của kiến trúc RISC-V. IP gồm một bộ đếm lên 64-bit được so sánh liên tục với giá trị so sánh 64-bit; khi hai giá trị bằng nhau, ngắt timer được tạo theo cấu hình của người dùng. IP được cấu hình qua bus APB (IP là APB slave).

Ứng dụng: tạo ngắt định kỳ (tick hệ điều hành), tạo trễ, tạo sự kiện, tạo xung/PWM, làm mốc thời gian (time base) cho hệ thống.

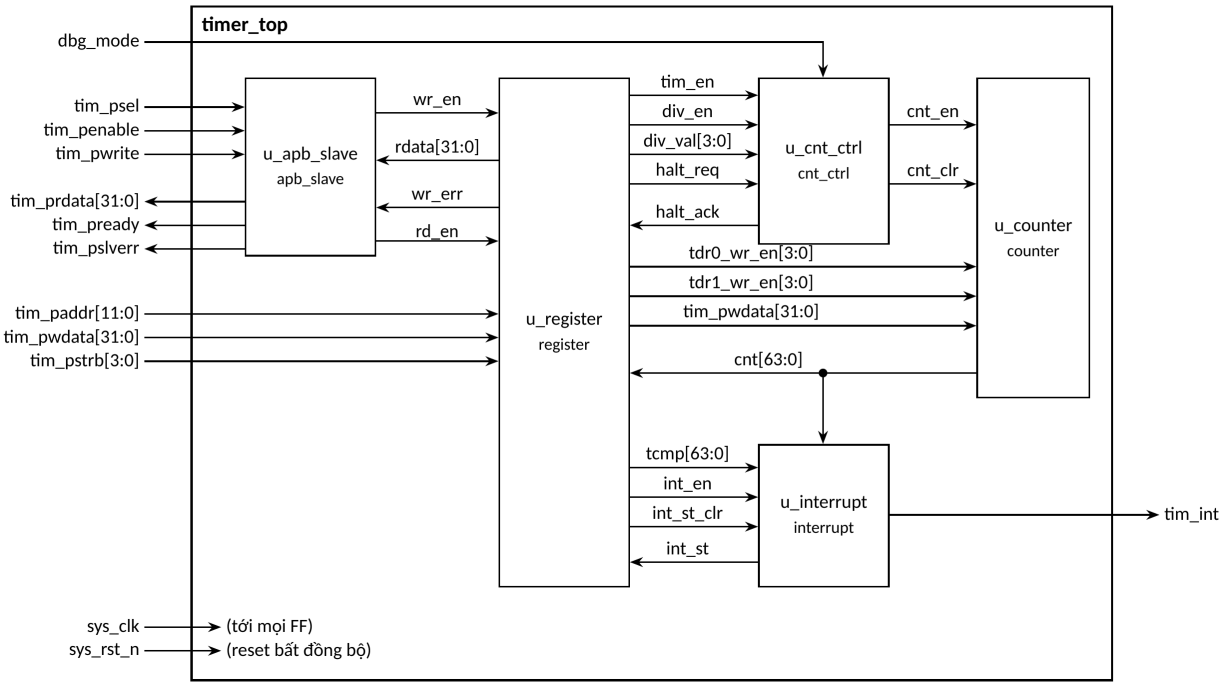
## 1.2. Tính năng chính

Timer IP hỗ trợ các tính năng sau (các mục đánh dấu [Advanced] là phần bổ sung của mức Advanced):

- **Bộ đếm lên 64-bit.** Bộ đếm gồm hai thanh ghi TDR0 (32 bit thấp) và TDR1 (32 bit cao). Phần mềm có thể đọc giá trị đếm bất kỳ lúc nào và có thể ghi (theo byte) để nạp giá trị ban đầu. Bộ đếm vẫn tiếp tục đếm khi xảy ra ngắt, và khi đạt giá trị lớn nhất thì tràn, quay về 0 rồi đếm tiếp.
- **Hai chế độ đếm.**
  - Chế độ mặc định (DIV_EN = 0): bộ đếm tăng 1 ở mỗi chu kỳ sys_clk.
  - Chế độ điều khiển (DIV_EN = 1): bộ đếm chỉ tăng 1 sau mỗi $2^{DIV_VAL}$ chu kỳ, tức tốc độ đếm được chia 1, 2, 4, ..., 256.
  - Đây không phải bộ chia tần số clock: toàn bộ IP vẫn chạy ở sys_clk, chỉ thời điểm bộ đếm được phép tăng là thay đổi.
- **Bật/tắt timer.** TIM_EN = 1 thì bộ đếm chạy, TIM_EN = 0 thì bộ đếm dừng. [Advanced] Khi TIM_EN chuyển từ 1 về 0, phần cứng tự xóa bộ đếm về 0; lần bật tiếp theo bộ đếm chạy lại từ 0 mà phần mềm không cần khởi tạo lại.
- **Ngắt timer.** Giá trị bộ đếm được so sánh liên tục với giá trị so sánh 64-bit {TCMP1, TCMP0}. Khi hai giá trị bằng nhau, bit chờ ngắt TISR.INT_ST được set; nếu TIER.INT_EN = 1 thì ngõ ra tim_int lên 1.
  - Ngắt là ngắt phần cứng, có thể che (maskable) và theo mức (level): tim_int giữ mức 1 cho đến khi phần mềm ghi 1 vào INT_ST để xóa, hoặc tắt INT_EN.
  - Tắt INT_EN chỉ che ngõ ra tim_int, không làm mất INT_ST; bật lại INT_EN thì ngắt xuất hiện lại nếu INT_ST chưa được xóa.
- **Giao tiếp APB.** IP là APB slave với địa chỉ 12-bit và dữ liệu 32-bit, gồm 8 thanh ghi. Đọc/ghi vào vùng địa chỉ reserved là RAZ/WI: đọc ra 0, ghi bị bỏ qua và không báo lỗi.
- [Advanced] **Wait state.** Mỗi giao dịch có đúng 1 chu kỳ chờ: tim_pready = 0 ở chu kỳ đầu của pha access và lên 1 ở chu kỳ kế tiếp, nên một giao dịch hoàn tất trong 3 chu kỳ (setup – wait – access).
- [Advanced] **Byte access.** tim_pstrb[3:0] chọn các byte được ghi; byte có strobe bằng 0 giữ nguyên giá trị cũ. Nhờ đó phần mềm có thể thay đổi một byte mà không ảnh hưởng các byte còn lại.
- [Advanced] **Error response.** Giao dịch ghi bị từ chối và trả tim_pslverr = 1 khi ghi giá trị bị cấm (lớn hơn 8) vào TCR.DIV_VAL, hoặc khi thay đổi DIV_EN/DIV_VAL trong lúc TIM_EN = 1. Khi có lỗi, cả giao dịch bị hủy, không bit nào của thanh ghi bị thay đổi.
- [Advanced] **Halt trong debug mode.** Khi dbg_mode = 1 và THCSR.HALT_REQ = 1, bộ đếm dừng lại và THCSR.HALT_ACK = 1 báo yêu cầu đã được chấp nhận. Khi xóa HALT_REQ, bộ đếm đếm tiếp từ đúng vị trí đã dừng (kể cả bộ chia div_cnt), nên số chu kỳ đếm thực tế của mỗi giá trị không đổi, không tính thời gian halt.
- **Clock và reset.** IP dùng một miền clock duy nhất sys_clk. Reset sys_rst_n bất đồng bộ, tích cực mức

thấp, đưa mọi thanh ghi về giá trị reset.

## 1.3. Sơ đồ khối




Hình 1: Sơ đồ khối của Timer IP

- **u_apb_slave**: xử lý giao dịch APB (wait state, error), tạo wr_en/rd_en gửi tới u_register; tim_prdata = rdata.
- **u_register**: giải mã địa chỉ, chứa các thanh ghi cấu hình, ghi theo byte, phát hiện lỗi (wr_err), mux đọc (rdata = 0 khi rd_en = 0).
- **u_cnt_ctrl**: tạo cnt_en từ TIM_EN/DIV_EN/DIV_VAL, xử lý halt, tạo cnt_clr khi TIM_EN H→L.
- **u_counter**: bộ đếm 64-bit; đếm, ghi bởi phần mềm, xóa bởi cnt_clr.
- **u_interrupt**: so sánh cnt với {TCMP1, TCMP0}, quản lý INT_ST và tạo tim_int.

## 1.4. Tín hiệu giao tiếp

Bảng 1: Tín hiệu giao tiếp của Timer IP

| Tên tín hiệu | Độ rộng | Hướng | Mô tả |
|---|---|---|---|
| sys_clk | 1 | Input | Clock hệ thống (cạnh lên). |
| sys_rst_n | 1 | Input | Reset bất đồng bộ, tích cực mức thấp. |
| tim_psel | 1 | Input | APB select. |
| tim_pwrite | 1 | Input | 1: ghi, 0: đọc. |
| tim_penable | 1 | Input | APB enable (pha access). |
| tim_paddr[11:0] | 12 | Input | Địa chỉ APB. |

| Tên tín hiệu | Độ rộng | Hướng | Mô tả |
|---|---|---|---|
| tim_pwdata[31:0] | 32 | Input | Dữ liệu ghi. |
| tim_prdata[31:0] | 32 | Output | Dữ liệu đọc, hợp lệ khi tim_pready = 1. |
| tim_pstrb[3:0] | 4 | Input | Write strobe: bit i = 1 thì ghi byte i. |
| tim_pready | 1 | Output | APB ready; 0 ở chu kỳ wait, 1 ở chu kỳ kết thúc. |
| tim_pslverr | 1 | Output | Báo lỗi giao dịch ghi (cùng chu kỳ tim_pready = 1). |
| tim_int | 1 | Output | Ngắt timer, tích cực High, theo mức. |
| dbg_mode | 1 | Input | Báo debug mode, dùng cho halt. Không đổi khi TIM_EN = 1. |

# 2. Đặc tả thanh ghi

## 2.1. Tổng hợp thanh ghi

Thanh ghi căn theo word. Địa chỉ không có trong bảng là reserved (RAZ/WI, không lỗi).

Bảng 2: Tổng hợp thanh ghi

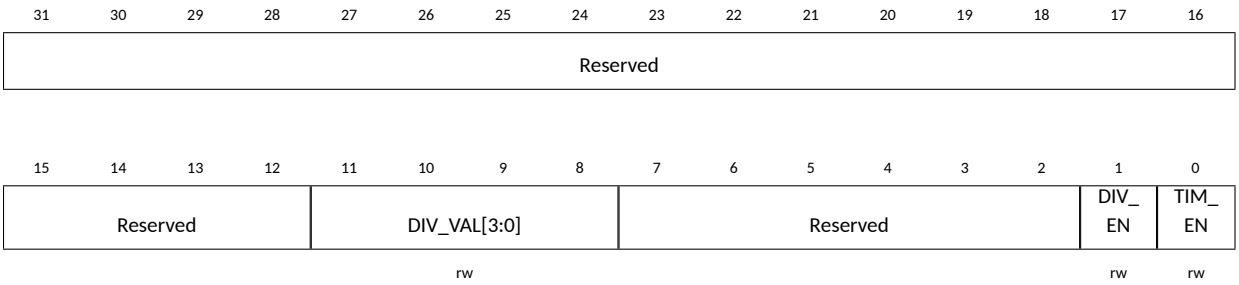
| Offset | Tên viết tắt | Tên thanh ghi |
|---|---|---|
| 0x00 | TCR | Timer Control Register |
| 0x04 | TDR0 | Timer Data Register 0 |
| 0x08 | TDR1 | Timer Data Register 1 |
| 0x0C | TCMP0 | Timer Compare Register 0 |
| 0x10 | TCMP1 | Timer Compare Register 1 |
| 0x14 | TIER | Timer Interrupt Enable Register |
| 0x18 | TISR | Timer Interrupt Status Register |
| 0x1C | THCSR | Timer Halt Control Status Register |
| Các địa chỉ khác | – | Reserved |

Kiểu bit: **RW** đọc/ghi (theo byte); **RO** chỉ đọc; **RW1C** ghi 1 để xóa.

## 2.2. Timer Control Register (TCR)

Địa chỉ offset: 0x0

Giá trị reset: 0x0000_0100



Bảng 3: Timer Control Register (TCR)

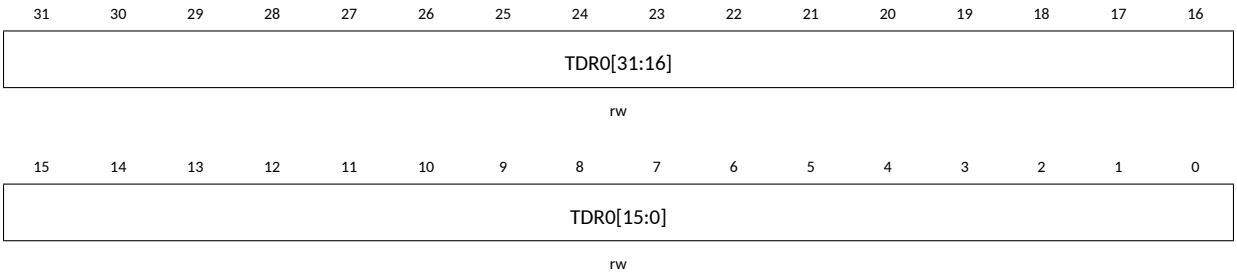
| Bit | Tên | Kiểu | Giá trị mặc định | Mô tả |
|---|---|---|---|---|
| 31:12 | Reserved | RO | 20'h0 | Reserved |
| 11:8 | DIV_VAL[3:0] | RW | 4'b0001 | Hệ số chia tốc độ đếm = $2^{DIV_VAL}$:<br>• 4'b0000 – 4'b1000: chia 1, 2, 4, ..., 256 (mặc định chia 2)<br>• Giá trị khác: bị cấm.<br>[Advanced] Ghi giá trị cấm hoặc thay đổi khi TIM_EN = 1: error response, TCR giữ nguyên. |
| 7:2 | Reserved | RO | 6'b0 | Reserved |
| 1 | DIV_EN | RW | 1'b0 | • 0: đếm theo sys_clk.<br>• 1: đếm theo DIV_VAL.<br>[Advanced] Thay đổi khi TIM_EN = 1: error response. |
| 0 | TIM_EN | RW | 1'b0 | • 0: bộ đếm không đếm.<br>• 1: bộ đếm đếm.<br>[Advanced] TIM_EN H→L: phần cứng xóa TDR0/TDR1. |

Lưu ý: chỉ báo lỗi khi giá trị thực sự thay đổi trên byte được strobe; ghi lại đúng giá trị cũ (ví dụ chỉ để xóa TIM_EN) là hợp lệ. Khi lỗi, cả giao dịch bị hủy.

## 2.3. Timer Data Register 0 (TDR0)

Địa chỉ offset: 0x4

Giá trị reset: 0x0000_0000



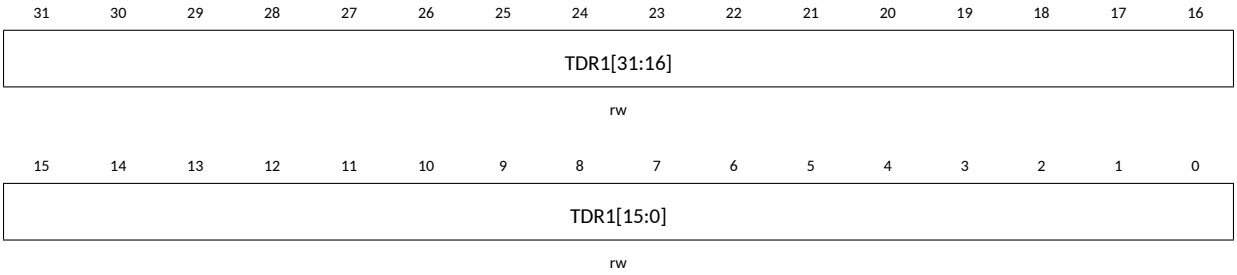

Bảng 4: Timer Data Register 0 (TDR0)

| Bit | Tên | Kiểu | Giá trị mặc định | Mô tả |
|---|---|---|---|---|
| 31:0 | TDR0 | RW | 32'h0000_0000 | 32-bit thấp của bộ đếm. [Advanced] Bị xóa khi TIM_EN H→L. |

## 2.4. Timer Data Register 1 (TDR1)

Địa chỉ offset: 0x8

Giá trị reset: 0x0000_0000



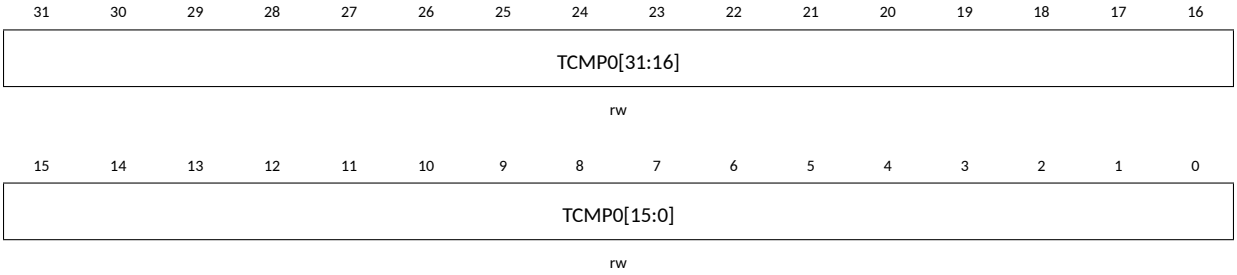

Bảng 5: Timer Data Register 1 (TDR1)

| Bit | Tên | Kiểu | Giá trị mặc định | Mô tả |
|---|---|---|---|---|
| 31:0 | TDR1 | RW | 32'h0000_0000 | 32-bit cao của bộ đếm. [Advanced] Bị xóa khi TIM_EN H→L. |

## 2.5. Timer Compare Register 0 (TCMP0)

Địa chỉ offset: 0xC

Giá trị reset: 0xFFFF_FFFF



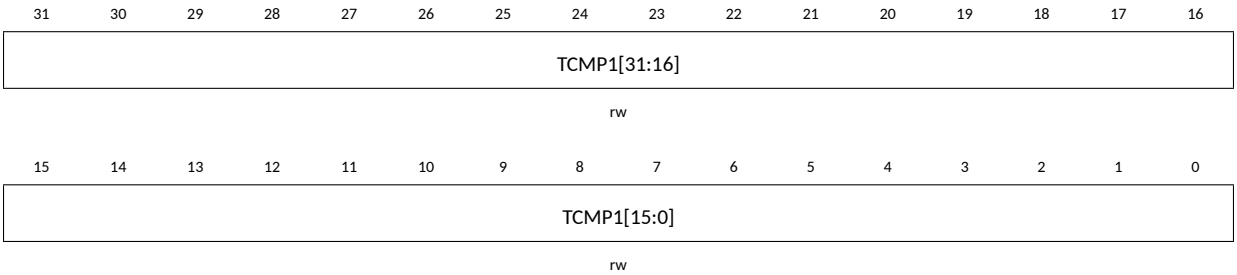

Bảng 6: Timer Compare Register 0 (TCMP0)

| Bit | Tên | Kiểu | Giá trị mặc định | Mô tả |
|---|---|---|---|---|
| 31:0 | TCMP0 | RW | 32'hFFFF_FFFF | 32-bit thấp của giá trị so sánh. |

## 2.6. Timer Compare Register 1 (TCMP1)

Địa chỉ offset: 0x10

Giá trị reset: 0xFFFF_FFFF



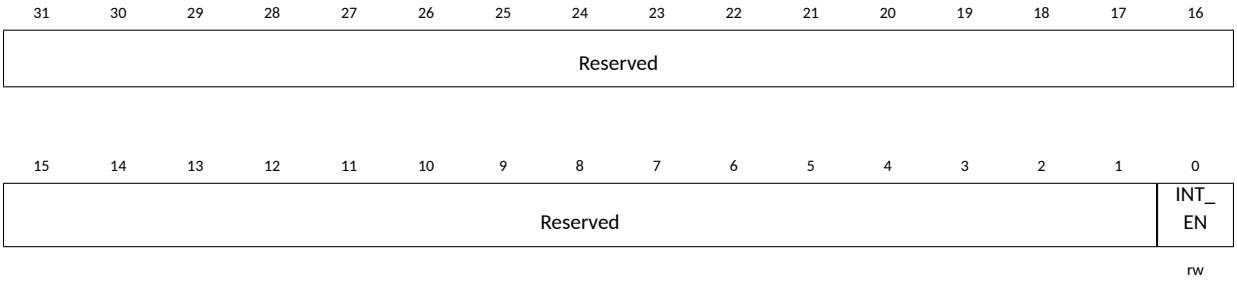
Bảng 7: Timer Compare Register 1 (TCMP1)

| Bit | Tên | Kiểu | Giá trị mặc định | Mô tả |
|---|---|---|---|---|
| 31:0 | TCMP1 | RW | 32'hFFFF_FFFF | 32-bit cao của giá trị so sánh. |

## 2.7. Timer Interrupt Enable Register (TIER)

Địa chỉ offset: 0x14

Giá trị reset: 0x0000_0000



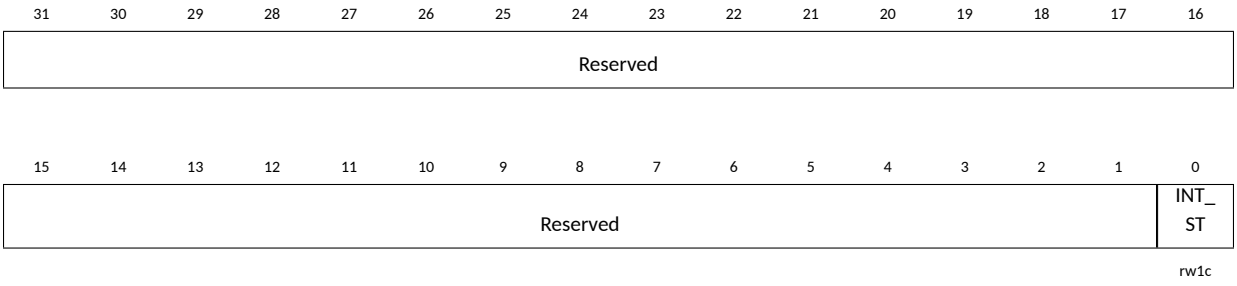

Bảng 8: Timer Interrupt Enable Register (TIER)

| Bit | Tên | Kiểu | Giá trị mặc định | Mô tả |
|---|---|---|---|---|
| 31:1 | Reserved | RO | 31'h0 | Reserved |
| 0 | INT_EN | RW | 1'b0 | • 0: che ngắt, tim_int = 0.<br>• 1: cho phép INT_ST ra tim_int.<br>Xóa INT_EN không làm thay đổi TISR.INT_ST. |

## 2.8. Timer Interrupt Status Register (TISR)

Địa chỉ offset: 0x18

Giá trị reset: 0x0000_0000




Bảng 9: Timer Interrupt Status Register (TISR)

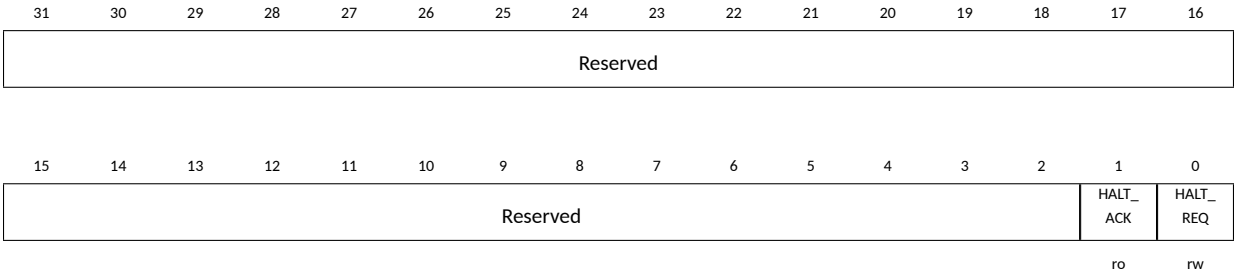
| Bit | Tên | Kiểu | Giá trị mặc định | Mô tả |
|---|---|---|---|---|
| 31:1 | Reserved | RO | 31'h0 | Reserved |

| | | | | |
|---|---|---|---|---|
| 0 | INT_ST | RW1C | 1'b0 | Bit chờ ngắt, set khi bộ đếm bằng giá trị so sánh. Ghi 1 để xóa; ghi 0 không tác dụng. Xóa có ưu tiên cao nhất. Bộ đếm vẫn đếm khi có ngắt. |

## 2.9. Timer Halt Control Status Register (THCSR)

Địa chỉ offset: 0x1C

Giá trị reset: 0x0000_0000




Bảng 10: Timer Halt Control Status Register (THCSR)

| Bit | Tên | Kiểu | Giá trị mặc định | Mô tả |
|---|---|---|---|---|
| 31:2 | Reserved | RO | 30'h0 | Reserved |
| 1 | HALT_ACK | RO | 1'b0 | 1: timer đang bị halt. HALT_ACK = dbg_mode & HALT_REQ. |
| 0 | HALT_REQ | RW | 1'b0 | 1: yêu cầu halt (chỉ có hiệu lực khi dbg_mode = 1). Xóa về 0 để đếm tiếp. |

# 3. Mô tả chức năng

Quy ước cho các sơ đồ logic trong mục này:

- Mỗi sơ đồ tương ứng với đúng một đoạn RTL (một khối always hoặc một nhóm assign). Không có khối logic nào được vẽ ở hai sơ đồ.
- Dây có nhãn ở mép trái là input của sơ đồ: hoặc là port của IP, hoặc là tín hiệu đã được tạo ở một sơ đồ khác (xem Bảng 11); dây có nhãn ở mép phải là output.
- Ô flip-flop ghi tên thanh ghi/field theo đặc tả và giá trị reset (rst). Mọi flip-flop dùng clock sys_clk và reset bất đồng bộ sys_rst_n (bubble ở đáy ô).
- Mux 2:1: ngõ 0 được chọn khi select = 0. Khi nhiều mux nối tiếp, mux càng gần flip-flop thì ưu tiên càng cao.

Bảng 11: Tín hiệu nội bộ và sơ đồ tạo ra chúng

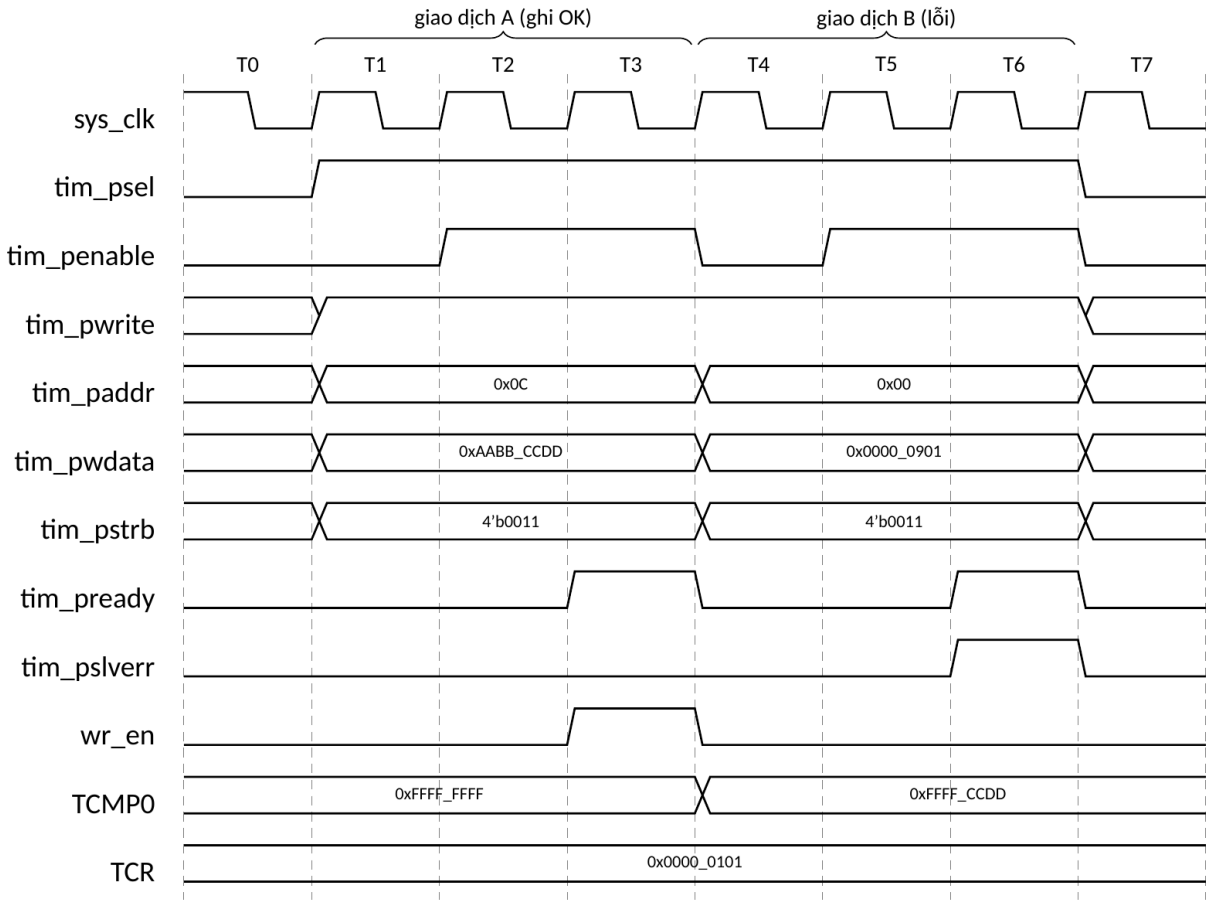
| Tín hiệu | Độ rộng | Tạo ở | Dùng ở |
|---|---|---|---|
| wr_en, rd_en | 1 | Hình 4 (APB slave) | Ghi thanh ghi / mux đọc |
| sel_tcr … sel_thcsr | 1 | Hình 6 (giải mã địa chỉ) | wr_err, các thanh ghi, tdr_wr_en, int_st_clr |
| wr_err | 1 | Hình 7 | APB slave |

| Tín hiệu | Độ rộng | Tạo ở | Dùng ở |
|---|---|---|---|
| tim_en, div_en, div_val[3:0] | 1/1/4 | Hình 8 (TCR) | wr_err, counter control, mux đọc |
| tcmp0[31:0], tcmp1[31:0] | 32 | Hình 9, 10 | Interrupt, mux đọc |
| tdr0_wr_en[3:0], tdr1_wr_en[3:0] | 4 | Hình 11 | cnt_wdata, counter |
| int_st_clr | 1 | Hình 12 | Interrupt |
| int_en, halt_req | 1 | Hình 13, 14 | Interrupt, halted, mux đọc |
| rdata[31:0] | 32 | Hình 15 (mux đọc) | APB slave |
| halted (= halt_ack) | 1 | Hình 18 | div_cnt, cnt_en, mux đọc |
| match | 1 | Hình 19 (div_cnt) | cnt_en |
| cnt_en, cnt_clr | 1 | Hình 20, 21 | Counter |
| cnt_wdata[63:0] | 64 | Hình 23 | Counter |
| cnt[63:0] | 64 | Hình 24 (counter) | cnt_wdata, interrupt, mux đọc |
| int_st | 1 | Hình 26 (interrupt) | Mux đọc |

## 3.1. APB slave

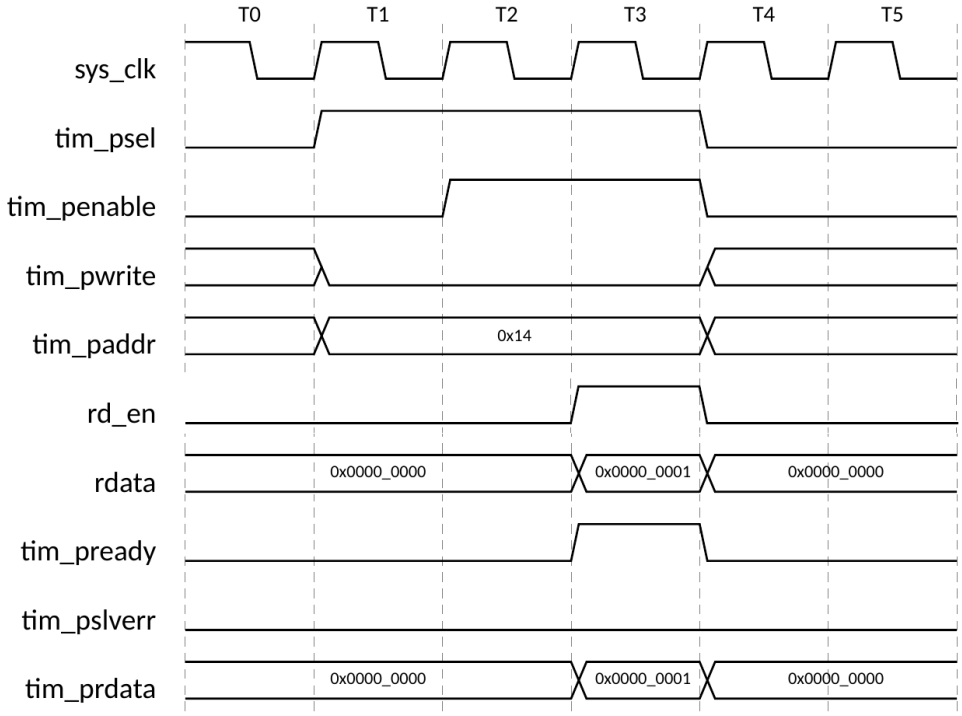
- Địa chỉ 12-bit (giải mã theo tim_paddr[11:2]), dữ liệu 32-bit, byte access qua tim_pstrb[3:0].
- 1 wait state: mỗi giao dịch gồm 3 chu kỳ (setup – wait – access).
- tim_pready, tim_pslverr lấy từ flip-flop; tim_prdata nối thẳng từ rdata (đọc ngay trong chu kỳ access).
- wr_en, rd_en được tổng hợp từ tim_psel, tim_penable, tim_pwrite và pready_q, gửi tới khối register.
- Ghi lỗi (wr_err = 1): tim_pslverr = 1, không tạo wr_en. Đọc không bao giờ lỗi.

**Dạng sóng**




Hình 2: Dạng sóng ghi APB slave (wait state, byte access, error response)

- Giao dịch A (T1 – T3): ghi TCMP0 với tim_pstrb = 4'b0011; wr_en ở T3, chỉ byte 0, 1 được cập nhật (0xFFFF_FFFF $\rightarrow$ 0xFFFF_CCDD).
- Giao dịch B (T4 – T6): ghi DIV_VAL = 9 (bị cấm); tim_pslverr = 1 ở T6, không có wr_en, TCR giữ nguyên.



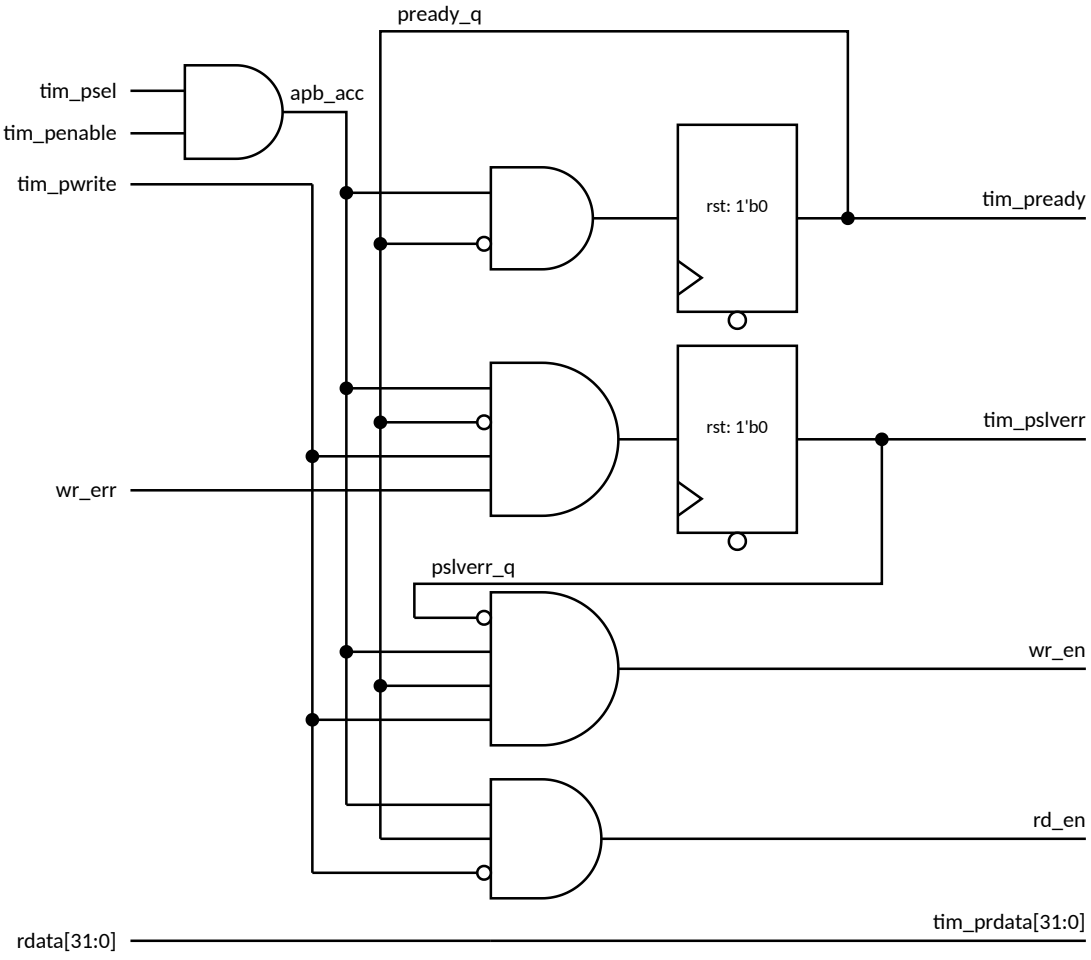

Hình 3: Dạng sóng đọc APB slave (wait state)

- T3 (access): rd_en = tim_pready = 1 nên rdata = tim_prdata = 0x0000_0001; ngoài T3 rdata = 0.

Bảng 12: Điều kiện tạo error response (ghi TCR)

| STT | Điều kiện | Biểu thức |
|---|---|---|
| 1 | DIV_VAL bị cấm ($>$ 8) | tim_pstrb[1] & (tim_pwdata[11:8] $>$ 8) |
| 2 | Đổi DIV_VAL khi TIM_EN = 1 | TIM_EN & tim_pstrb[1] & (tim_pwdata[11:8] != DIV_VAL) |
| 3 | Đổi DIV_EN khi TIM_EN = 1 | TIM_EN & tim_pstrb[0] & (tim_pwdata[1] != DIV_EN) |

**Sơ đồ logic**




Hình 4: Sơ đồ logic APB slave

Giải thích sơ đồ logic:

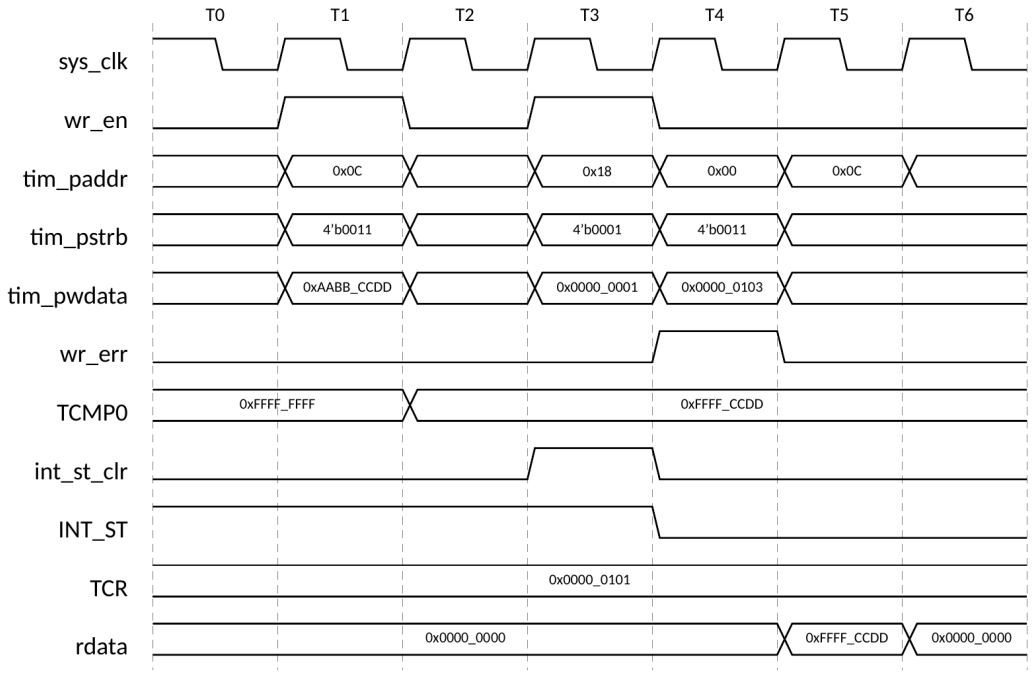
- **Nhận biết pha access:** cổng AND đầu tiên gộp tim_psel và tim_penable thành apb_acc. apb_acc = 1 nghĩa là master đang ở pha access và chờ slave trả lời.
- **Tạo wait state (tim_pready):** flip-flop pready_q nhận apb_acc AND với pready_q đảo. Ở chu kỳ đầu của pha access pready_q còn bằng 0, nên ở cạnh clock kế tiếp FF lên 1 và tim_pready = 1 kết thúc giao dịch. Trong chu kỳ tim_pready = 1, chân đảo làm cổng AND ra 0 nên FF tự về 0 ở chu kỳ sau. Nhờ vậy mọi giao dịch có đúng 1 chu kỳ chờ.
- **Báo lỗi (tim_pslverr):** trong chu kỳ chờ của giao dịch ghi (apb_acc = 1, pready_q = 0, tim_pwrite = 1), nếu wr_err = 1 thì flip-flop pslverr_q được set. tim_pslverr vì thế lên 1 đúng ở chu kỳ tim_pready = 1 và tự về 0 ở chu kỳ sau.
- **Cho phép ghi (wr_en):** wr_en chỉ bằng 1 ở chu kỳ cuối của giao dịch ghi (apb_acc, pready_q, tim_pwrite đều bằng 1) và khi không có lỗi (chân pslverr_q đảo). Giao dịch lỗi không bao giờ tạo wr_en, nên không có bit nào bị ghi.
- **Cho phép đọc (rd_en):** giống wr_en nhưng cho giao dịch đọc (tim_pwrite = 0), cũng chỉ bật ở chu kỳ cuối.

- **Dữ liệu đọc:** rdata nối thẳng ra tim_prdata, không qua flip-flop, nên dữ liệu có ngay trong chu kỳ tim_pready = 1.

## 3.2. Register

- Giải mã địa chỉ tạo 8 tín hiệu chọn sel_xxx, dùng chung cho mọi thanh ghi.
- Ghi byte i của một thanh ghi khi wr_en = 1, sel_xxx = 1 và tim_pstrb[i] = 1.
- Tạo wr_err (truy cập TCR bị cấm), tdr0/1_wr_en[3:0] cho counter, int_st_clr cho interrupt và mux đọc rdata (có điều kiện rd_en).

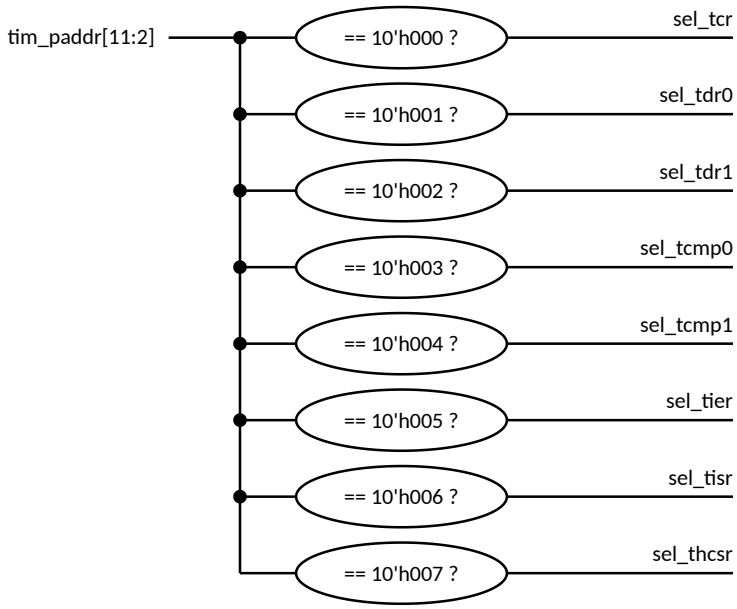
**Dạng sóng**




Hình 5: Dạng sóng ghi/đọc thanh ghi

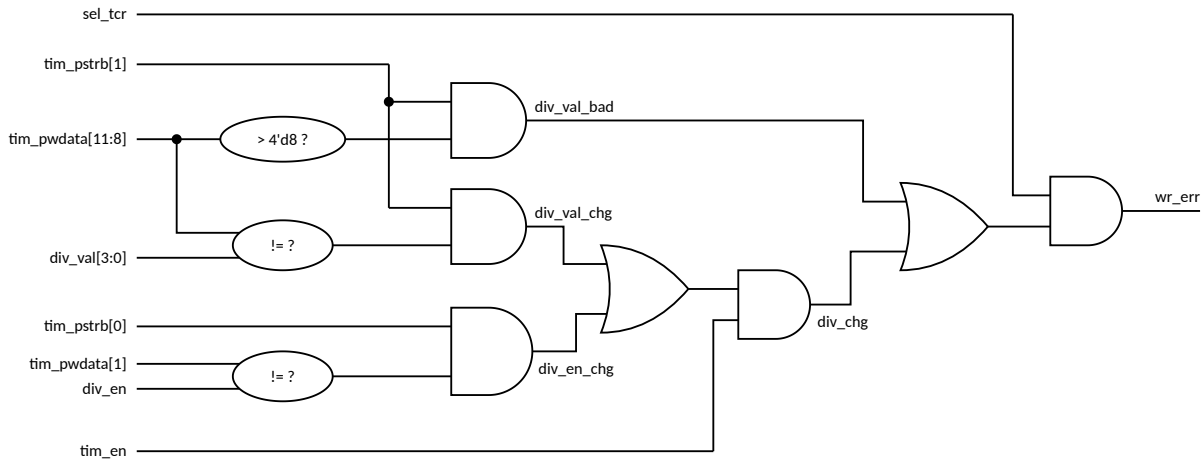
- T1: ghi byte 0, 1 của TCMP0. T3: ghi 1 vào TISR, INT_ST bị xóa.
- T4: đổi DIV_EN khi TIM_EN = 1 → wr_err = 1, TCR giữ nguyên. T5: đọc TCMP0, rdata = 0xFFFF_CCDD; các chu kỳ không đọc rdata = 0.

**Sơ đồ logic**




Hình 6: Sơ đồ logic giải mã địa chỉ

Giải thích: mỗi bộ so sánh đối chiếu word address tim_paddr[11:2] với offset của một thanh ghi (offset ≫ 2) và bật tín hiệu chọn tương ứng. Tại một thời điểm có tối đa một sel_xxx bằng 1; địa chỉ reserved không bật sel nào nên ghi bị bỏ qua (WI). Các sơ đồ phía sau chỉ dùng sel_xxx, không so sánh địa chỉ lại.



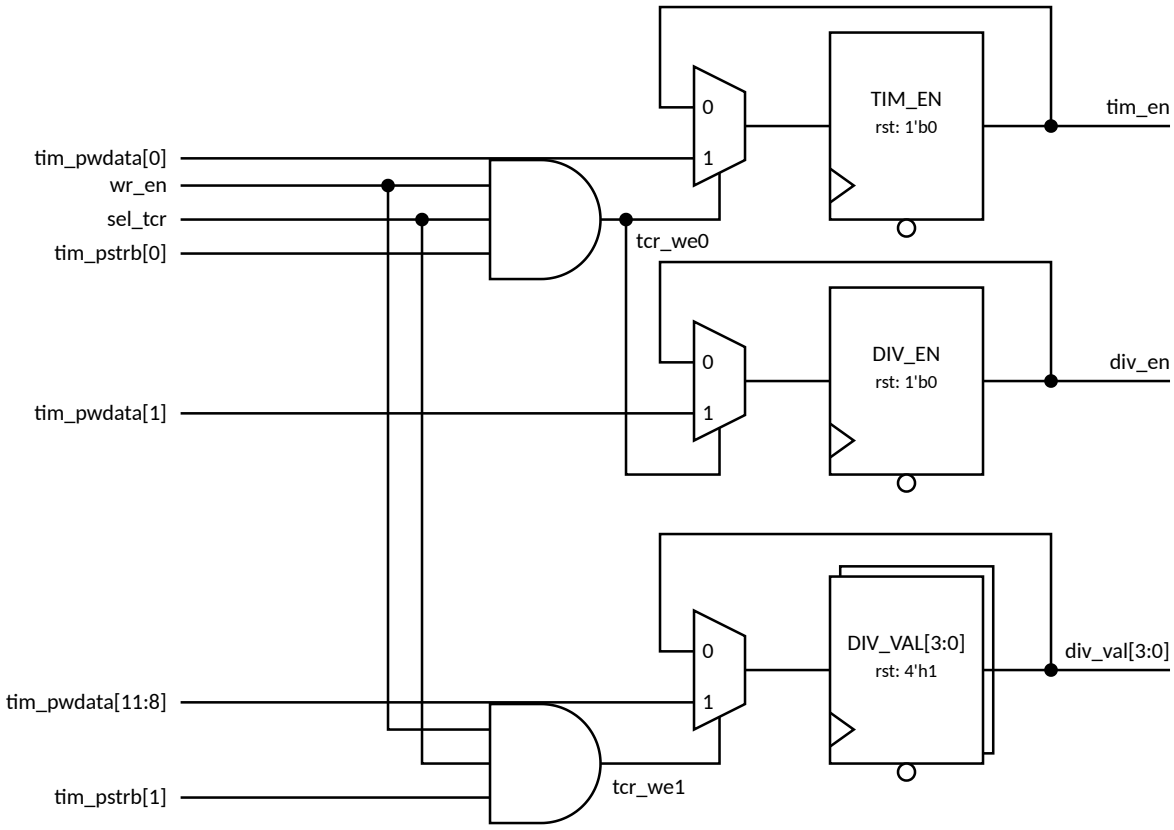

Hình 7: Sơ đồ logic phát hiện lỗi truy cập TCR (wr_err)

Giải thích sơ đồ logic phát hiện lỗi:

- **Giá trị bị cấm (div_val_bad):** khi byte 1 được ghi (tim_pstrb[1] = 1) và giá trị mới của DIV_VAL (tim_-

pwdata[11:8]) lớn hơn 8.

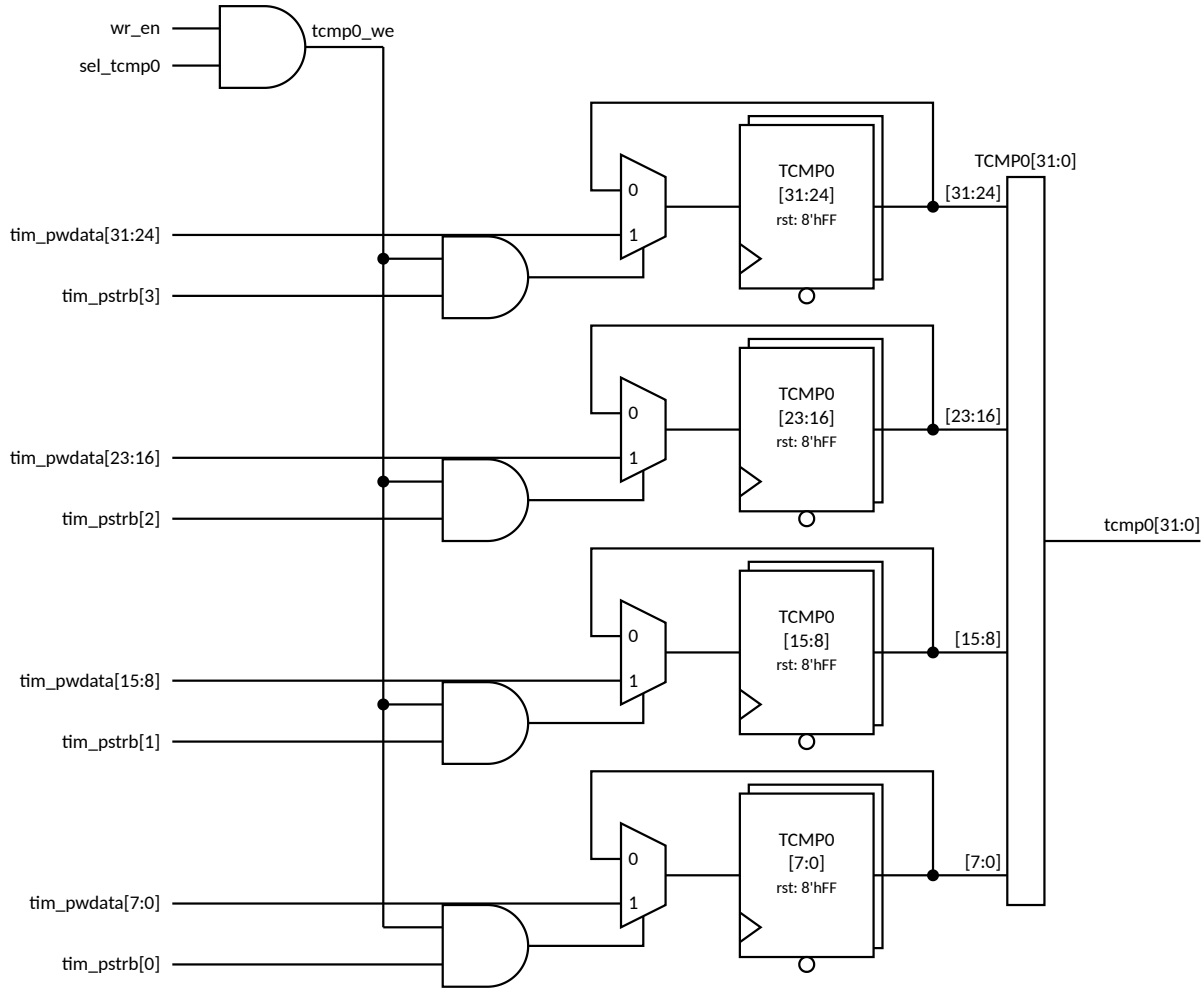
- **Thay đổi field (div_val_chg, div_en_chg):** bộ so sánh "!=" đối chiếu giá trị mới với giá trị đang lưu (div_val, div_en); mỗi kết quả chỉ có tác dụng khi byte chứa field đó được strobe. Hai kết quả được OR lại rồi AND với tim_en thành div_chg, nên việc thay đổi chỉ bị cấm khi timer đang chạy.
- **Kết quả (wr_err):** div_val_bad OR div_chg, AND với sel_tcr: chỉ TCR có thể báo lỗi. Ghi lại đúng giá trị đang có (ví dụ chỉ để xóa TIM_EN) không tạo lỗi. wr_err là tổ hợp; APB slave chỉ lấy nó trong chu kỳ chờ của giao dịch ghi.



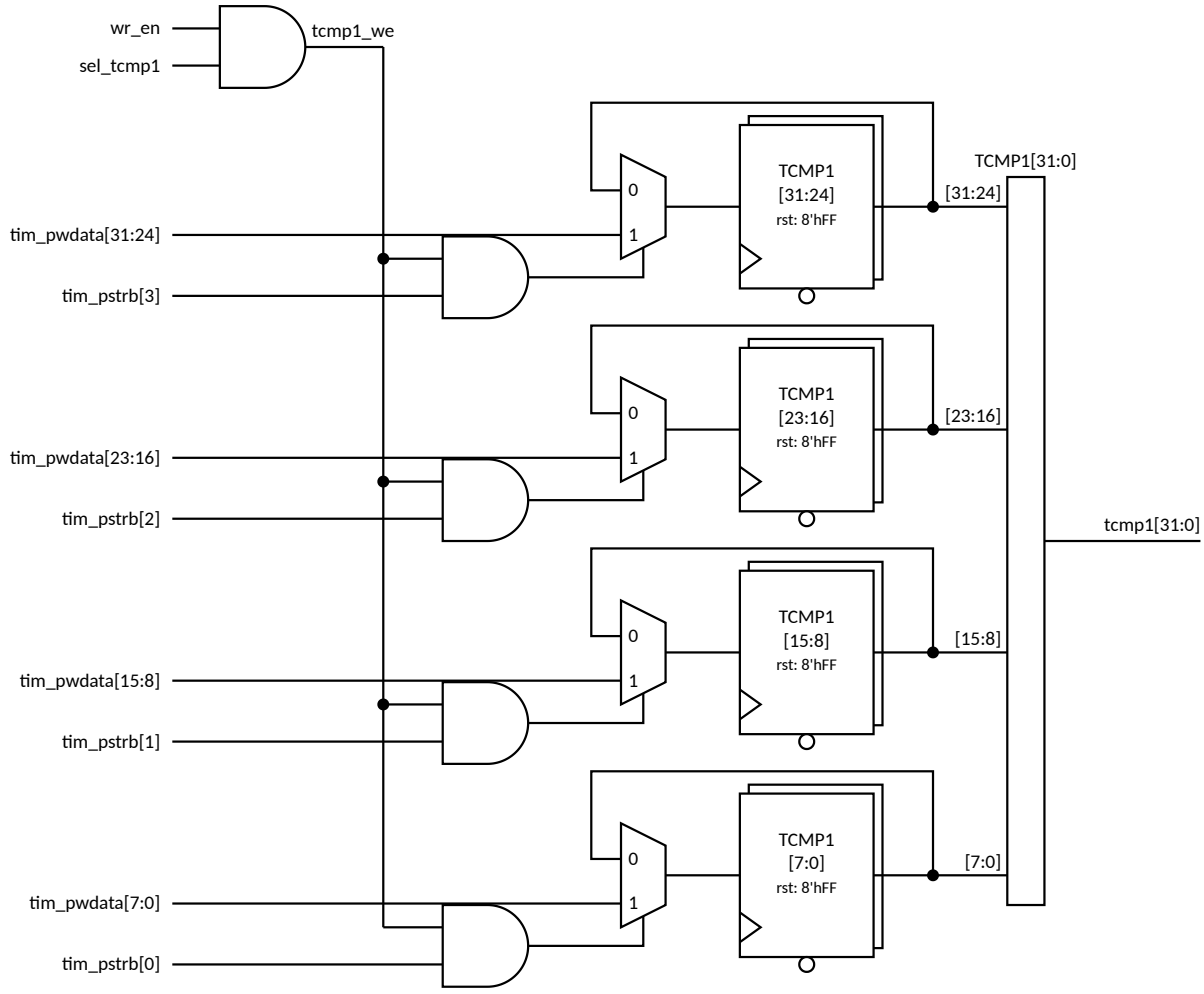

Hình 8: Sơ đồ logic thanh ghi TCR

Giải thích sơ đồ logic TCR:

- **Cho phép ghi theo byte:** tcr_we0 = wr_en AND sel_tcr AND tim_pstrb[0] cho phép ghi byte 0 (TIM_EN từ tim_pwdata[0], DIV_EN từ tim_pwdata[1]); tcr_we1 dùng tim_pstrb[1] để ghi byte 1 (DIV_VAL từ tim_pwdata[11:8]).
- **Giữ giá trị:** mỗi flip-flop đứng sau một mux 2:1. Khi tín hiệu cho phép bằng 1, mux chọn dữ liệu mới (ngõ 1); ngược lại mux đưa giá trị hiện tại của FF quay lại (ngõ 0).
- **Không ghi khi lỗi:** khi giao dịch lỗi, APB slave không tạo wr_en nên TCR giữ nguyên. Reset: TIM_EN = 0, DIV_EN = 0, DIV_VAL = 4'h1.



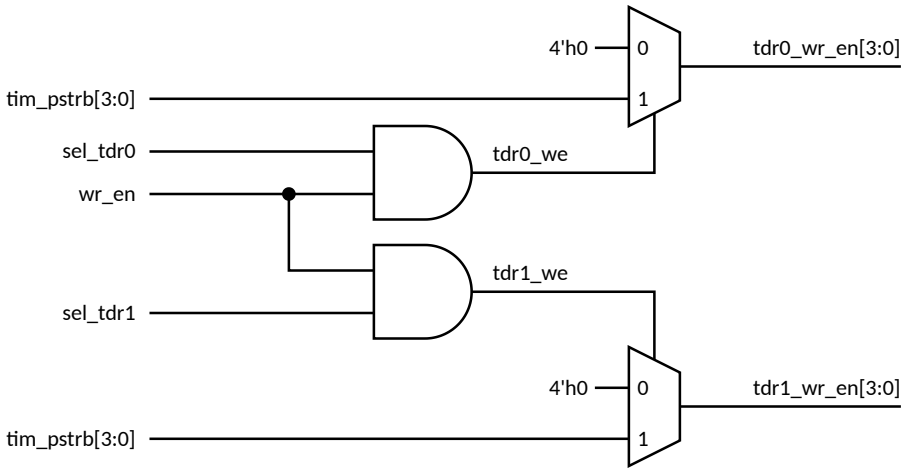

Hình 9: Sơ đồ logic thanh ghi TCMP0




Hình 10: Sơ đồ logic thanh ghi TCMP1

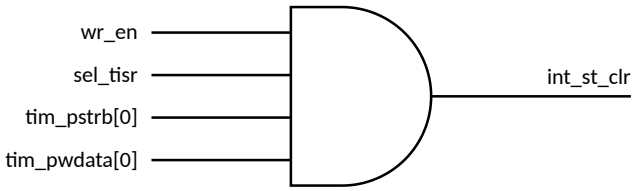
Giải thích sơ đồ logic TCMP0/TCMP1:

- **Chọn thanh ghi:** cổng AND đầu tiên tạo tcmp0_we (tcmp1_we) khi có wr_en và sel_tcmp0 (sel_tcmp1).
- **Chọn byte:** với mỗi byte i, tcmpX_we AND tim_pstrb[i] làm select cho mux của byte đó. Byte có strobe bằng 1 nạp phần tương ứng của tim_pwdata; byte có strobe bằng 0 giữ nguyên nhờ đường phản hồi.
- **Gộp kết quả:** 4 byte được gộp thành tcmp0[31:0] (tcmp1[31:0]) gửi tới khối interrupt và mux đọc. Mọi byte reset về 8'hFF.



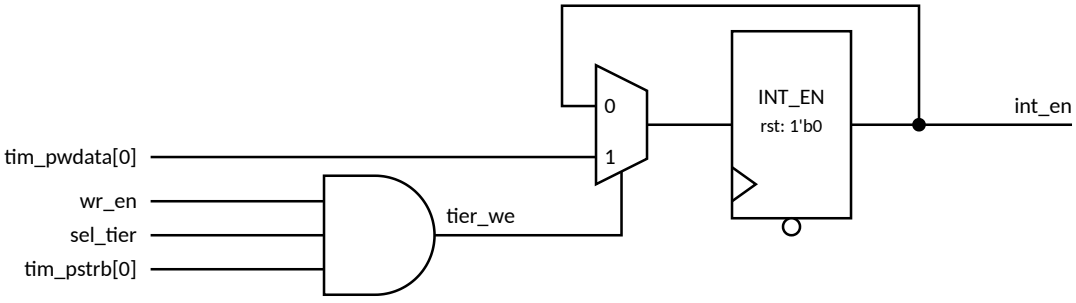

Hình 11: Sơ đồ logic tdr0_wr_en/tdr1_wr_en

Giải thích: TDR0/TDR1 nằm trong khối counter, nên khối register chỉ tạo tín hiệu cho phép ghi theo byte. Khi wr_en = 1 và sel_tdr0 = 1, mux đưa nguyên tim_pstrb[3:0] ra tdr0_wr_en[3:0] (mỗi bit cho phép ghi một byte); các trường hợp khác ra 4'h0. TDR1 làm tương tự với sel_tdr1.

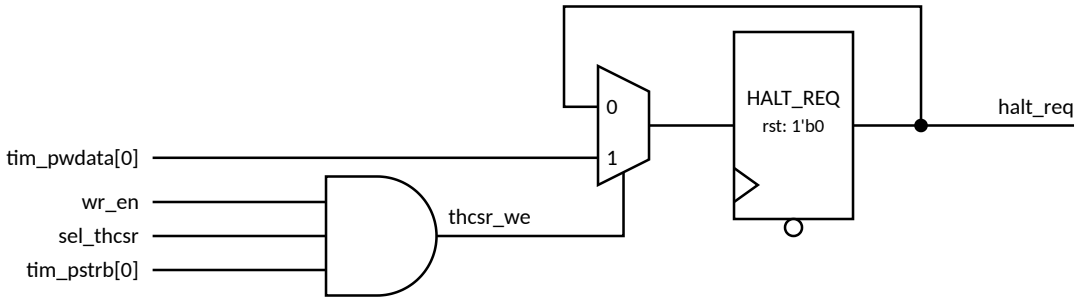



Hình 12: Sơ đồ logic int_st_clr

Giải thích: int_st_clr bằng 1 trong chu kỳ ghi vào TISR có byte 0 được strobe và bit 0 của dữ liệu bằng 1. Đây là xung 1 chu kỳ gửi tới khối interrupt. Ghi 0 không tạo xung, nên INT_ST có hành vi ghi 1 để xóa (RW1C).

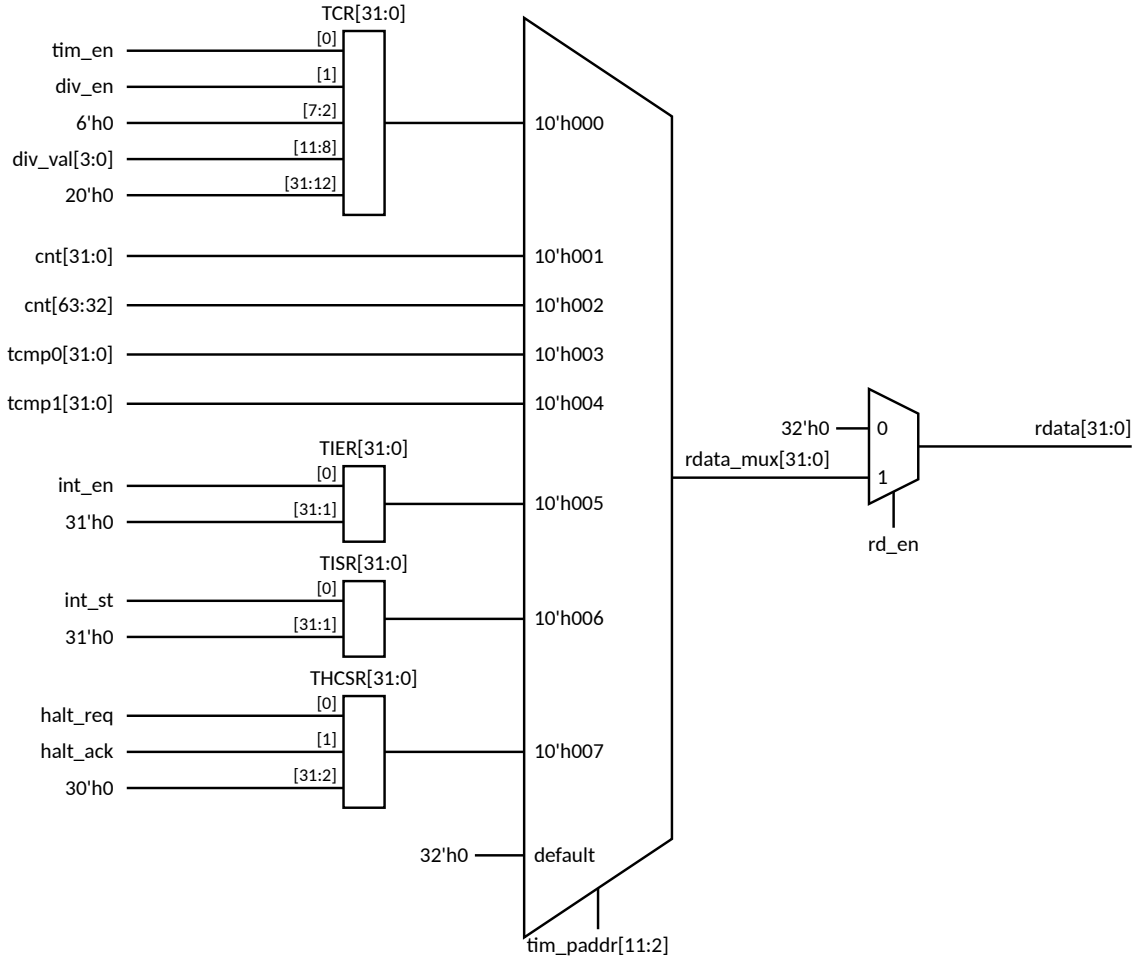



Hình 13: Sơ đồ logic thanh ghi TIER (INT_EN)




Hình 14: Sơ đồ logic thanh ghi THCSR (HALT_REQ)

Giải thích sơ đồ logic TIER/THCSR: mỗi bit là một flip-flop 1 bit kèm mux giữ giá trị. Khi ghi đúng thanh ghi (sel_tier hoặc sel_thcsr) với byte 0 được strobe, FF nạp tim_pwdata[0]; các trường hợp khác FF giữ nguyên. Cả hai reset về 0. HALT_ACK là bit chỉ đọc, lấy từ halted (Hình 18), nên không có flip-flop ở đây.




Hình 15: Sơ đồ logic mux đọc

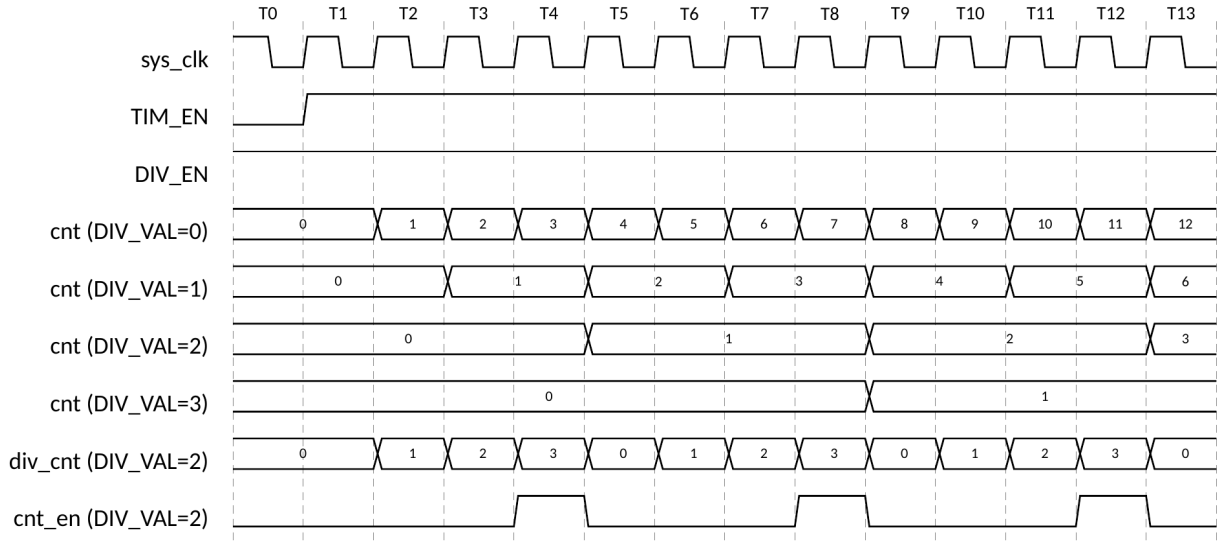
Giải thích sơ đồ logic mux đọc:

- **Ghép word đọc:** các field của mỗi thanh ghi được ghép thành một word 32-bit theo đúng vị trí bit trong đặc tả; bit reserved được gán 0. TDR0/TDR1 lấy trực tiếp từ cnt[31:0]/cnt[63:32].
- **Mux địa chỉ:** mux lớn chọn một trong 8 word theo tim_paddr[11:2], kết quả là rdata_mux. Địa chỉ reserved rơi vào nhánh default và trả về 0 (RAZ).
- **Mux rd_en:** mux 2:1 đặt sau mux địa chỉ và có ưu tiên cao hơn. Chỉ khi rd_en = 1, rdata_mux mới được đưa ra rdata; các chu kỳ khác rdata = 0.

## 3.3. Counter control

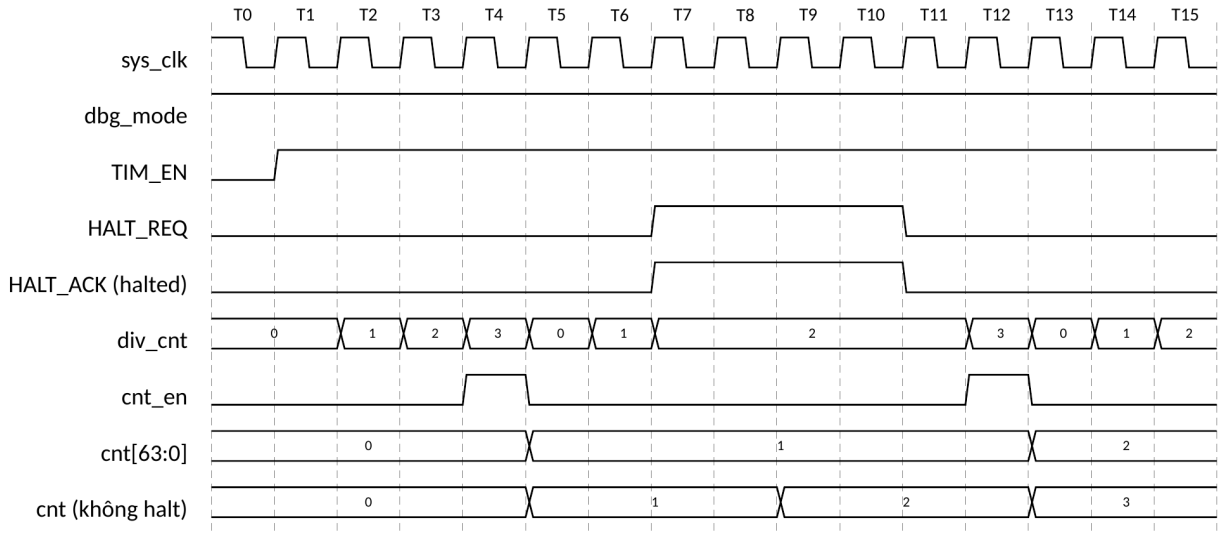
- DIV_EN = 0: đếm mỗi chu kỳ. DIV_EN = 1: đếm mỗi $2^{\text{DIV_VAL}}$ chu kỳ (không phải chia clock).
- Halt khi dbg_mode & HALT_REQ: div_cnt và bộ đếm cùng dừng.
- Tạo cnt_clr khi TIM_EN H$\rightarrow$L.

**Dạng sóng**




Hình 16: Dạng sóng chế độ đếm

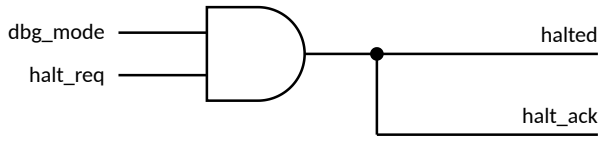
- DIV_VAL = 0/1/2/3: bộ đếm tăng sau mỗi 1/2/4/8 chu kỳ; cnt_en là xung khi div_cnt = limit.




Hình 17: Dạng sóng halt trong debug mode (DIV_VAL = 2)

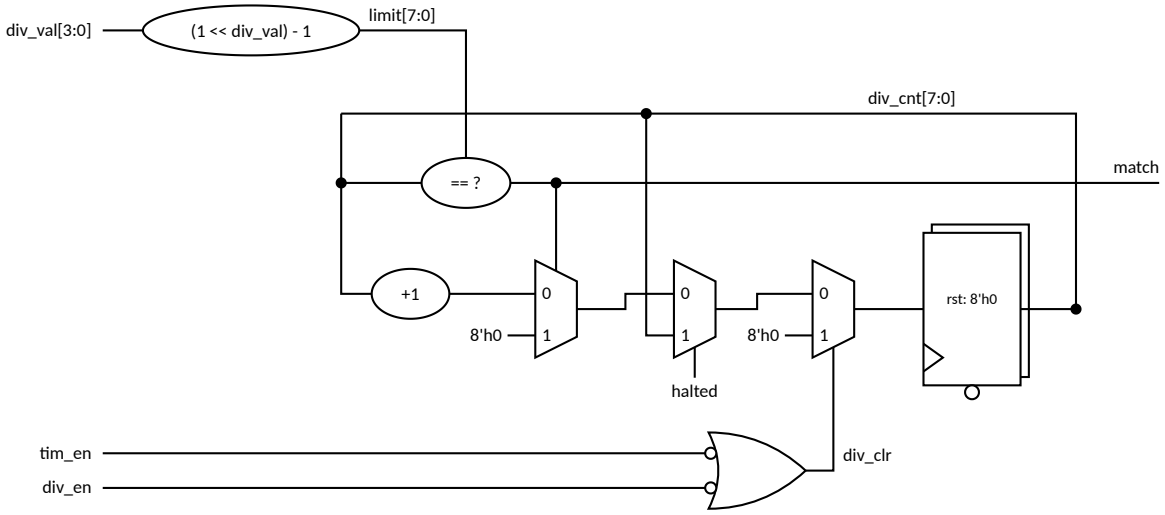
- T7 – T10: halt, div_cnt và cnt giữ nguyên. Sau khi xóa HALT_REQ, đếm tiếp từ đúng vị trí đã dừng: cnt chỉ trễ đúng 4 chu kỳ halt so với khi không halt (cnt = 2 ở T13 thay vì T9).

**Sơ đồ logic**




Hình 18: Sơ đồ logic halted/halt_ack

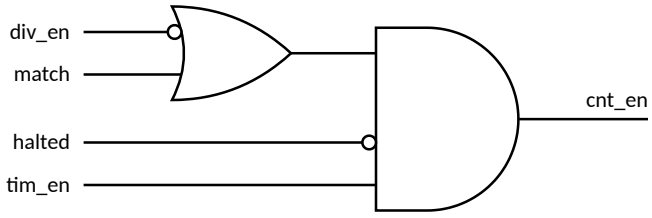
Giải thích: halted = 1 chỉ khi hệ thống ở debug mode (dbg_mode = 1) và phần mềm đã yêu cầu halt (halt_req = 1). Cùng tín hiệu này được đọc ra qua THCSR.HALT_ACK để phần mềm biết yêu cầu đã được chấp nhận. Khi xóa HALT_REQ hoặc thoát debug mode, halted về 0 và bộ đếm chạy lại.




Hình 19: Sơ đồ logic bộ đếm chia div_cnt

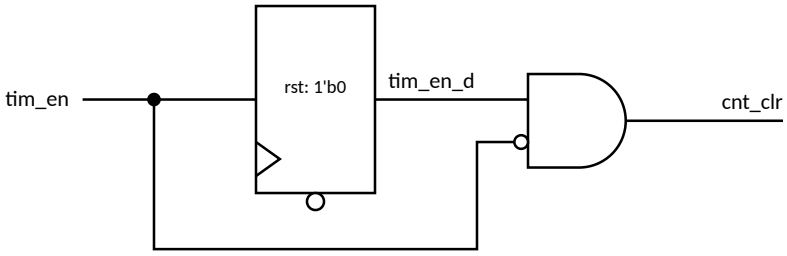
Giải thích sơ đồ logic div_cnt:

- **Giới hạn chia:** limit = $2^{\text{div_val}} - 1$, tức 0, 1, 3, ..., 255 với div_val từ 0 đến 8 (tính ở 9 bit rồi lấy 8 bit thấp).
- **So sánh:** match = 1 khi div_cnt bằng limit, tức đã đủ $2^{\text{div_val}}$ chu kỳ.
- **Giá trị kế tiếp của div_cnt** đi qua 3 mux nối tiếp:
  - Mux đầu: match = 1 thì quay về 0, ngược lại tăng 1. div_cnt vì vậy đếm vòng $0 \rightarrow$ limit.
  - Mux giữa: halted = 1 thì giữ nguyên, nên vị trí trong chu kỳ chia được bảo toàn trong lúc halt.
  - Mux cuối (ưu tiên cao nhất): div_clr = 1 (tim_en = 0 hoặc div_en = 0) thì xóa về 0, để mỗi lần bật lại chu kỳ chia bắt đầu từ đầu.




Hình 20: Sơ đồ logic cnt_en

Giải thích: cổng OR với chân div_en đảo cho phép đếm ở mọi chu kỳ khi ở chế độ mặc định (div_en = 0), còn ở chế độ điều khiển chỉ cho phép khi match = 1. Kết quả được AND với tim_en và halted đảo. Như vậy cnt_en là xung 1 chu kỳ lặp lại sau mỗi $2^{DIV_VAL}$ chu kỳ, và bằng 0 khi timer tắt hoặc đang halt.



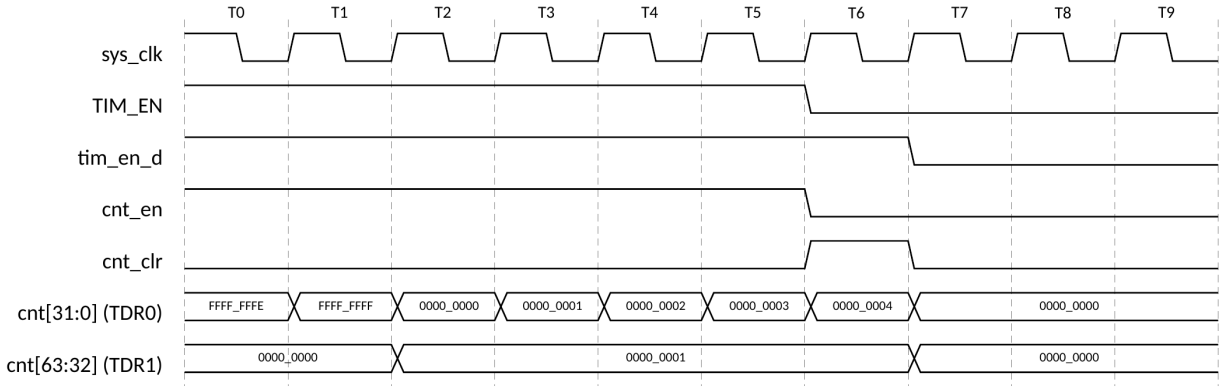

Hình 21: Sơ đồ logic phát hiện cạnh xuống TIM_EN (cnt_clr)

Giải thích: flip-flop tim_en_d lưu giá trị tim_en của chu kỳ trước. Khi chu kỳ trước tim_en = 1 mà hiện tại tim_en = 0 (timer vừa bị tắt), cổng AND tạo xung cnt_clr dài 1 chu kỳ để khối counter xóa bộ đếm về 0.

## 3.4. Counter

- Bộ đếm 64-bit cnt = {TDR1, TDR0}, tăng khi cnt_en = 1, quay về 0 khi tràn.
- Ghi theo byte qua TDR0/TDR1; xóa về 0 khi cnt_clr = 1.

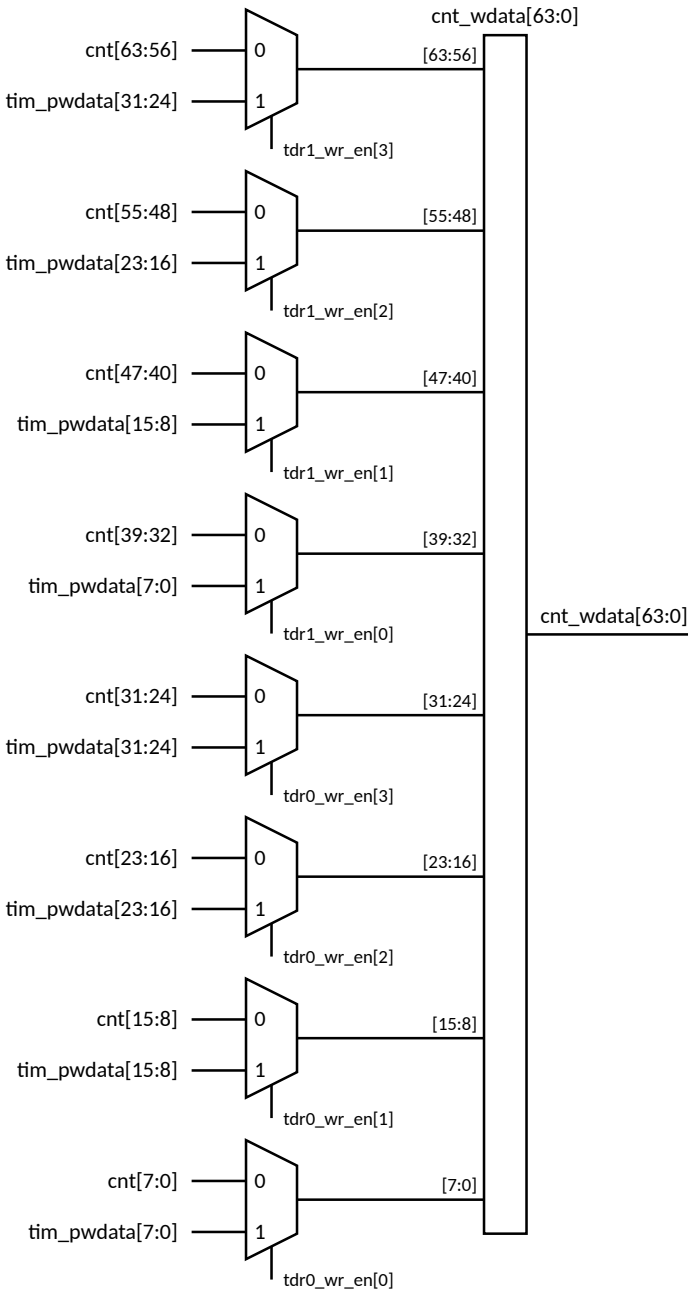
### Dạng sóng




Hình 22: Dạng sóng bộ đếm

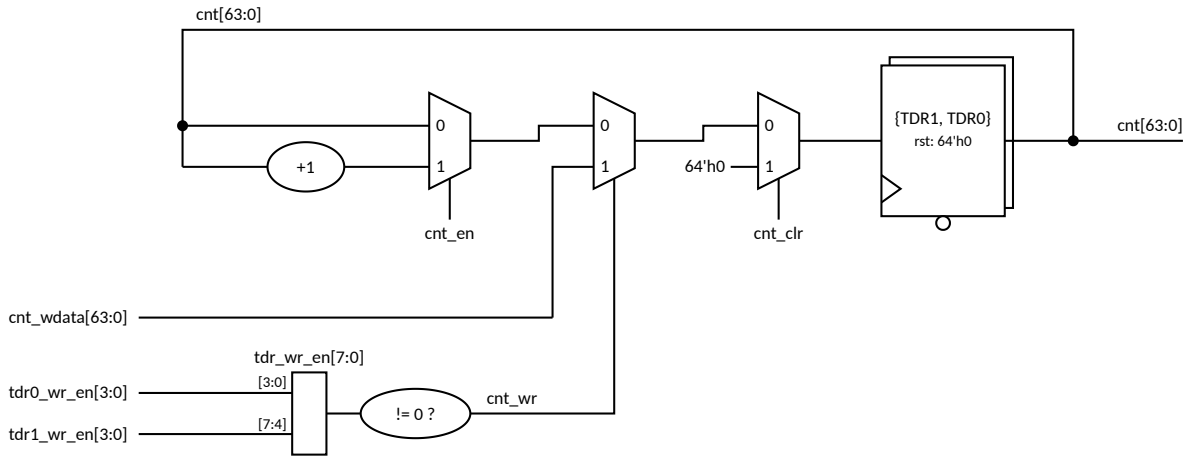
- TDR0 tràn từ 0xFFFF_FFFF thì TDR1 tăng 1. TIM_EN xuống Low ở T6 thì cnt_clr xóa cnt về 0.

**Sơ đồ logic**




Hình 23: Sơ đồ logic dữ liệu ghi bộ đếm cnt_wdata

Giải thích: cnt_wdata là giá trị 64-bit mà bộ đếm sẽ nhận khi phần mềm ghi. Mỗi byte có một mux 2:1: byte có bit write enable tương ứng bằng 1 (tdr0_wr_en[i] cho byte 0–3, tdr1_wr_en[i] cho byte 4–7) lấy byte của tim_pwdata; byte còn lại giữ giá trị cnt hiện tại. Nhờ vậy ghi TDR0 không làm thay đổi TDR1 và ngược lại, và ghi một byte không ảnh hưởng các byte khác.




Hình 24: Sơ đồ logic bộ đếm 64-bit

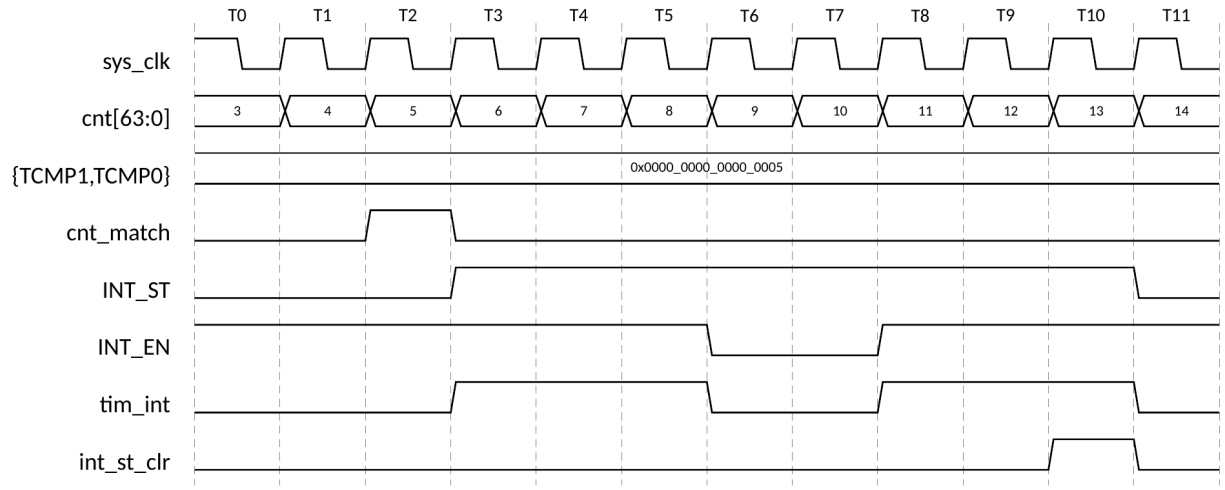
Giải thích sơ đồ logic counter:

- **Phát hiện ghi:** 8 bit write enable được gộp thành tdr_wr_en[7:0]; cnt_wr = 1 khi có bất kỳ bit nào bằng 1.
- **Chuỗi mux theo độ ưu tiên:**
  - Mux đầu: cnt_en = 1 thì chọn cnt + 1, ngược lại giữ nguyên. Khi cnt đạt giá trị lớn nhất, bộ cộng tự quay về 0 (tràn).
  - Mux giữa: cnt_wr = 1 thì chọn cnt_wdata. Phần mềm ghi có ưu tiên cao hơn việc đếm.
  - Mux cuối (ưu tiên cao nhất): cnt_clr = 1 thì nạp 64'h0.
- **Ngõ ra:** flip-flop 64-bit {TDR1, TDR0} reset về 0; cnt[63:0] được đưa tới mux đọc, khối interrupt và cnt_-wdata.

## 3.5. Interrupt

- INT_ST set khi cnt == {TCMP1, TCMP0}, xóa bằng ghi 1 (ưu tiên cao nhất).
- tim_int = INT_ST & INT_EN (ngắt theo mức, maskable).

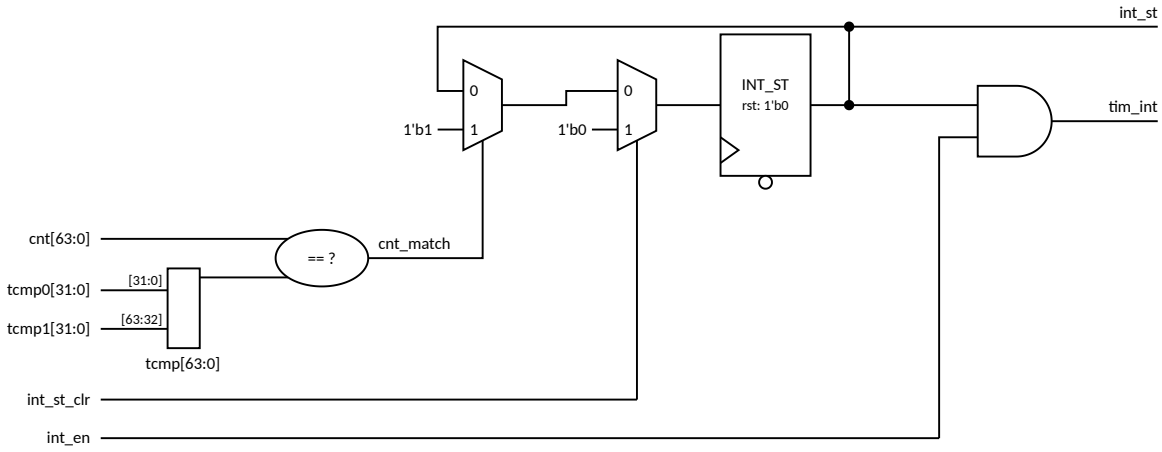
### Dạng sóng




Hình 25: Dạng sóng ngắt timer

- T2: cnt = {TCMP1, TCMP0} = 5 nên cnt_match = 1; INT_ST và tim_int lên 1 từ T3.
- T6 – T7: INT_EN = 0 che tim_int, INT_ST vẫn giữ. T10: int_st_clr = 1, INT_ST và tim_int về 0 từ T11.

### Sơ đồ logic




Hình 26: Sơ đồ logic khối interrupt (INT_ST, tim_int)

Giải thích sơ đồ logic:

- **So sánh:** tcmp1 và tcmp0 được gộp thành tcmp[63:0]; bộ so sánh 64-bit cho cnt_match = 1 trong chu kỳ cnt bằng tcmp.
- **Bit chờ ngắt INT_ST:** mux đầu set INT_ST lên 1 khi cnt_match = 1, ngược lại giữ nguyên. Mux sau (gần FF, ưu tiên cao hơn) xóa INT_ST về 0 khi int_st_clr = 1, nên nếu set và xóa xảy ra cùng chu kỳ thì xóa thắng. Sau khi được set, INT_ST giữ 1 cho đến khi phần mềm xóa.

- **Ngõ ra ngắt:** tim_int = 1 khi INT_ST = 1 và int_en = 1. Khi int_en = 0, ngõ ra bị che nhưng INT_ST vẫn giữ, nên bật lại INT_EN thì ngắt xuất hiện lại nếu INT_ST chưa được xóa. int_st được đưa tới mux đọc (TISR).

## 4. Lịch sử

| Ngày | Phiên bản | Mô tả | Tác giả |
|---|---|---|---|
| 2026-10-19 | 1.0 | Tạo mới (mức Standard) | Ngô Hồ Quân |
| 2026-10-24 | 2.0 | Nâng cấp lên mức Advanced: wait state, error response, byte access, halt trong debug mode, tự xóa bộ đếm khi TIM_EN H→L | Ngô Hồ Quân |